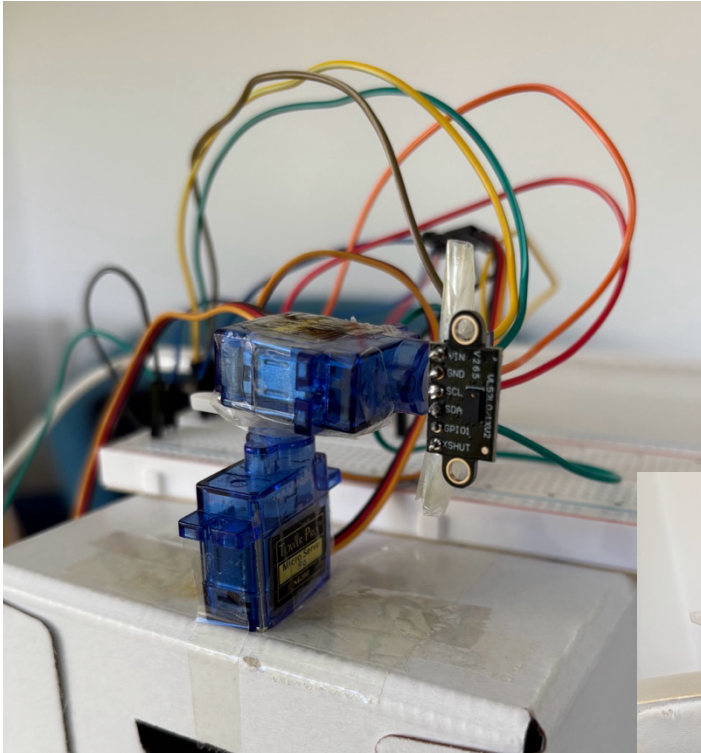

Home-Made Room Scanning & Visualisation System

Summer Project Report



Steven Robert Carswell
Summer 2026

1. Introduction

This report documents the development of a homemade 2D & 3D room scanning system built using an ESP32 microcontroller, servo motor, and distance sensing hardware. The project was undertaken as a personal engineering project with the aim of producing a point cloud reconstruction of room geometry using low-cost hobbyist equipment.

The 2D system works by mounting a distance sensor on a servo motor, sweeping it across a horizontal arc, and logging the angle and distance readings at each step via serial communication to a Python environment. The collected data is then processed and visualised as a 2D Cartesian point cloud, with automated wall detection and line fitting applied to reconstruct room geometry.

The project involved working with both hardware and software, beginning with an HC-SR04 ultrasonic sensor before realising its limitations and upgrading to a VL53L1X time-of-flight laser sensor. Throughout the process there were challenges including sensor noise, multipath reflections, and beam width limitations which were attempted to fix through software filtering techniques.

The report follows the full development journey chronologically, covering hardware selection, programming, data collection, visualisation development, and attempts at 3D room reconstruction.

2. Acknowledgements

I would like to acknowledge the use of Claude (Anthropic) for help with a lot of the code and assistance throughout this project.

3. *Hardware Selection*

Below is a list of hardware used throughout this project:

Microcontroller

- ESP32 Development Board

Sensors

- HC-SR04 Ultrasonic Sensor (initial sensor used)
- TF-Luna LiDAR Range Finder (attempted upgrade - hardware failure)
- VL53L0X V2 Time-of-Flight Laser Distance Sensor (final sensor)

Motors

- SG90 Micro Servo Motor

Electronic components

- Breadboard
- Jumper Wires
- Resistors

Software Tools

- Arduino IDE (ESP32 code)
- Jupyter Notebook (data collection and visualisation)
- Python Libraries: NumPy, Matplotlib, Pandas, PySerial, SciPy

4. System Overview

The scanning system developed throughout this project was consistent across all the hardware used. At its core, a distance sensor is mounted on a servo motor controlled by an ESP32 microcontroller. The servo sweeps the sensor across a horizontal arc, pausing at defined angle increments to take distance measurements before sending the data over to the connected computer.

On the software side, a Python script running in Jupyter Notebook listens to the serial port in real time, recording the incoming angle and distance pairs and logging them. Once a full sweep is complete, a separate visualisation script loads, applies simple filtering and cleaning, then converts the polar coordinates to Cartesian, and renders the point cloud using Matplotlib or Plotly.

This process (sensing and actuation on the ESP32, serial data transmission, Python logging and visualisation) remained consistent throughout the project. Hardware upgrades and software improvements were made but the fundamental system was kept the same at each stage.

The key stages are:

1. Sensing - distance sensor measures range at each servo angle
2. Transmission - ESP32 sends angle and distance
3. Logging - Python records incoming data to CSV in real time
4. Processing - outlier removal and filtering applied to the raw data
5. Visualisation - cleaned data converted to Cartesian coordinates and plotted

5. HC-SR04 Ultrasonic Sensor - First Prototype

5.1 Wiring and Hardware Setup

The HC-SR04 ultrasonic sensor was selected as the initial sensing component because of its low cost and straightforward interface with the ESP32 microcontroller. The sensor operates by emitting an ultrasonic pulse and measuring the time taken for the echo to return, converting this duration to a distance measurement in centimetres. The overall subject matter that is trying to be scanned and visualised is the corner of a room, something that is easily noticeable for a simple scanner.

The sensor was mounted on an SG90 servo motor and wired to the ESP32 as shown in Figure 1. The servo was programmed to sweep from 25° to 165° in 1° increments, pausing at each angle for 250ms to allow measurements to be taken. To improve data reliability, 10 samples were averaged at each angle position with individual readings outside the range of 5–400cm discarded before averaging. Figure 1 shows the set-up at this stage:

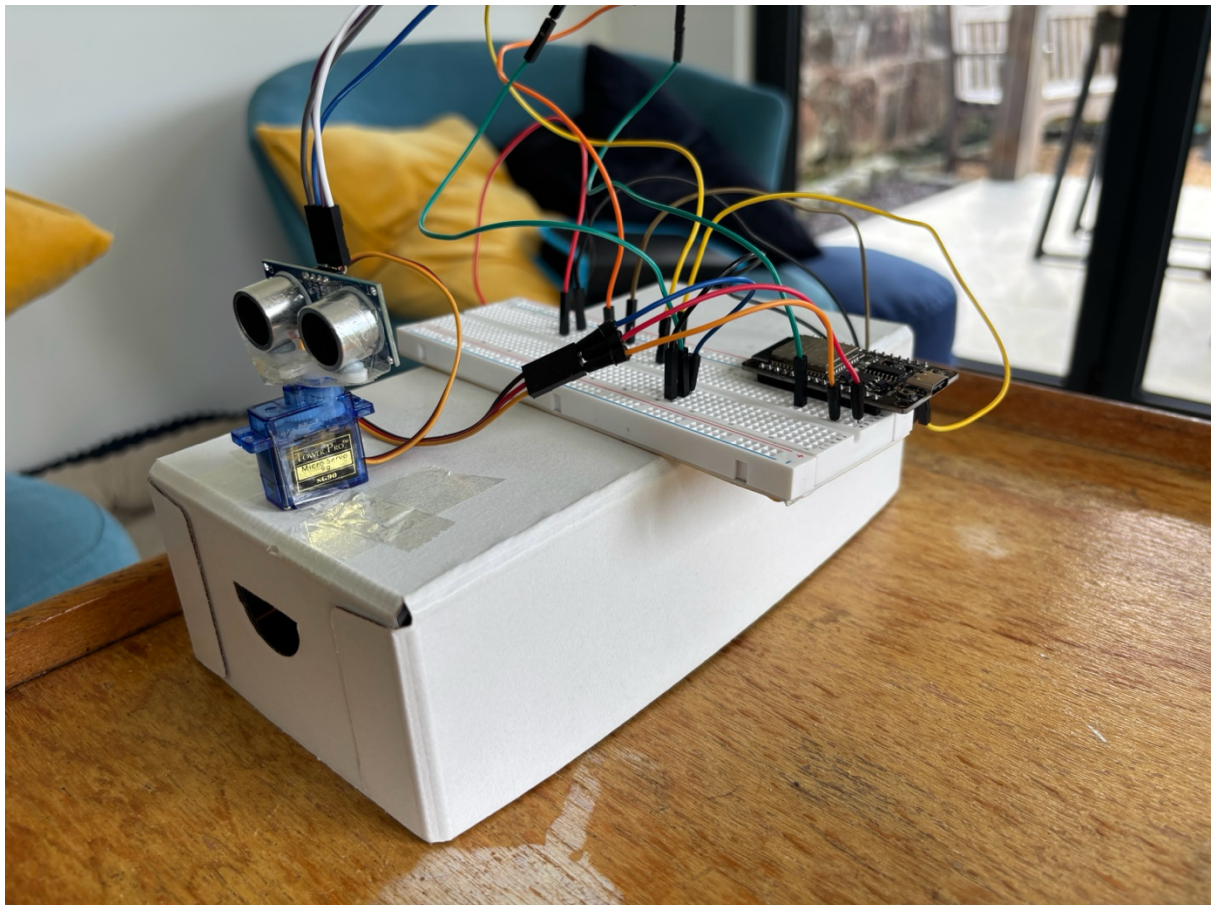


Figure 1

5.2 Arduino Code

The Arduino code was written in C++ using the Arduino IDE and uploaded to the ESP32 microcontroller. The code was structured around two key functions - a distance measurement function and a main sweep loop.

The measurement function fired the HC-SR04 trigger pin, measured the duration of the returning echo pulse using the `pulseIn()` function, and converted the duration to a distance in centimetres using the speed of sound.

As stated earlier, to improve reliability a second averaging function called this measurement repeatedly, collecting 10 samples per angle position and discarding any readings outside the valid range of 5–400cm before returning the mean of the valid readings.

The main sweep loop incremented the servo position from 25° to 165° in 1° steps, waiting 250ms at each position for the servo to mechanically settle before triggering a measurement. The resulting angle and distance pair was then transmitted over serial in a comma separated format:

```
Serial.print(angle);  
Serial.print(",");  
Serial.println(distance);
```

A SCAN COMPLETE marker was transmitted at the end of each full sweep, allowing the Python logging script to detect when a complete dataset had been collected.

Here is the full code used at this point:

```
#include <ESP32Servo.h>  
  
#define TRIG_PIN 5  
#define ECHO_PIN 18  
#define SERVO_PIN 13  
  
Servo scannerServo;  
  
// Scan settings  
const int startAngle = 25;  
const int endAngle = 165;  
const int stepAngle = 1; // smaller = more detailed  
const int samplesPerAngle = 10; // more = smoother data  
const int settleDelay = 250; // wait after servo moves  
  
float measureDistance() {  
    digitalWrite(TRIG_PIN, LOW);  
    delayMicroseconds(2);
```

```

digitalWrite(TRIG_PIN, HIGH);
delayMicroseconds(10);
digitalWrite(TRIG_PIN, LOW);

long duration = pulseIn(ECHO_PIN, HIGH, 30000);

if (duration == 0) {
    return -1;
}

return duration * 0.0343 / 2.0; // cm
}

float averageDistance() {
    float total = 0;
    int validCount = 0;

    for (int i = 0; i < samplesPerAngle; i++) {
        float d = measureDistance();

        if (d > 2 && d < 400) {
            total += d;
            validCount++;
        }

        delay(40);
    }

    if (validCount == 0) {
        return -1;
    }

    return total / validCount;
}

void setup() {
    Serial.begin(115200);

    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);

    scannerServo.attach(SERVO_PIN);
    delay(1000);

    Serial.println("angle,distance_cm");
}

void loop() {
    for (int angle = startAngle; angle <= endAngle; angle += stepAngle) {
        scannerServo.write(angle);
        delay(settleDelay);
    }
}

```

```
float distance = averageDistance();

Serial.print(angle);
Serial.print(",");
Serial.println(distance);
}

Serial.println("SCAN_COMPLETE");

delay(3000);
}
```

5.3 Python Code

Data collection was handled by a Python script running in a Jupyter Notebook environment, using the PySerial library to interface with the ESP32 over the USB serial connection at 115200 baud. The script opened a CSV file for writing and continuously read incoming serial lines, recording the angle and distance values for each reading received.

Each valid reading was written to the python script in real time as the servo swept, with the angle and distance stored as separate columns. A try/except block was used to discard any non-data lines such as the SCAN_COMPLETE marker or any corrupted serial data without interrupting the logging process.

The resulting data contained two columns, angle and distance in cm, forming the raw dataset used as the input for the visualisation. This approach of logging directly to python script allowed the data to be inspected, filtered, and replotted freely of the scanning hardware, making it straightforward to adjust on the visualisation without needing to re-run the physical scan.

The visualisation script loaded the data using the Pandas library, applied numeric conversion and validity filtering, and converted the polar coordinate pairs to Cartesian x and y positions using NumPy before plotting the resulting point cloud using Matplotlib.

Here is the full code:

```
import serial
import csv

ser = serial.Serial('/dev/cu.usbserial-10', 115200)

with open("scan.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow(["angle", "distance"])

    while True:
        line = ser.readline().decode().strip()

        try:
            angle, distance = line.split(",")

            writer.writerow([angle, distance])

            print(angle, distance)

        except:
            pass
```

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# — Load & clean —————
data = pd.read_csv("scan.csv")
data["angle"] = pd.to_numeric(data["angle"], errors="coerce")
data["distance"] = pd.to_numeric(data["distance"], errors="coerce")
data = data.dropna()
data = data[data["distance"] > 0]
data = data[data["distance"] < 200]

low, high = data["distance"].quantile(0.02), data["distance"].quantile(0.98)
data = data[(data["distance"] > low) & (data["distance"] < high)]
data = data.sort_values("angle").reset_index(drop=True)

# Rolling median filter
rolling_med = data["distance"].rolling(window=11, center=True, min_periods=1).median()
data = data[np.abs(data["distance"] - rolling_med) < 8]
data = data.reset_index(drop=True)

# — Polar → Cartesian —————
angles = np.radians(data["angle"])
distances = data["distance"]
x = distances * np.cos(angles)
y = distances * np.sin(angles)

# — Plot —————
fig, ax = plt.subplots(figsize=(10, 10))
fig.patch.set_facecolor("white")
ax.set_facecolor("white")

ax.scatter(x, y, c="#378ADD", s=8, alpha=0.6, zorder=3)
ax.scatter(0, 0, s=200, marker="+", color="black", linewidths=2, zorder=5)

ax.set_xlabel("X Position (cm)", color="black")
ax.set_ylabel("Y Position (cm)", color="black")
ax.set_title("2D Room Scan", color="black", fontsize=14, pad=12)
ax.tick_params(colors="black")
ax.spines[:].set_color("#cccccc")
ax.grid(True, color="#eeeeee", linewidth=0.5)
ax.axis("equal")

plt.tight_layout()
plt.savefig("/Users/stevencarswell/Desktop/scan_clean.png", dpi=300, facecolor="white")
plt.show()

```

5.4 Results

Initial scan results, shown in Figure 2, produced a point cloud with a distinctive curved arc shape rather than the expected straight wall geometry. This turned out to be a consequence of the sensor's 15° conical beam width. As the sensor rotates, the beam strikes flat walls at varying angles of incidence, producing a $1/\cos(\theta)$ distance relationship that shows as a curve in Cartesian space rather than a straight line.

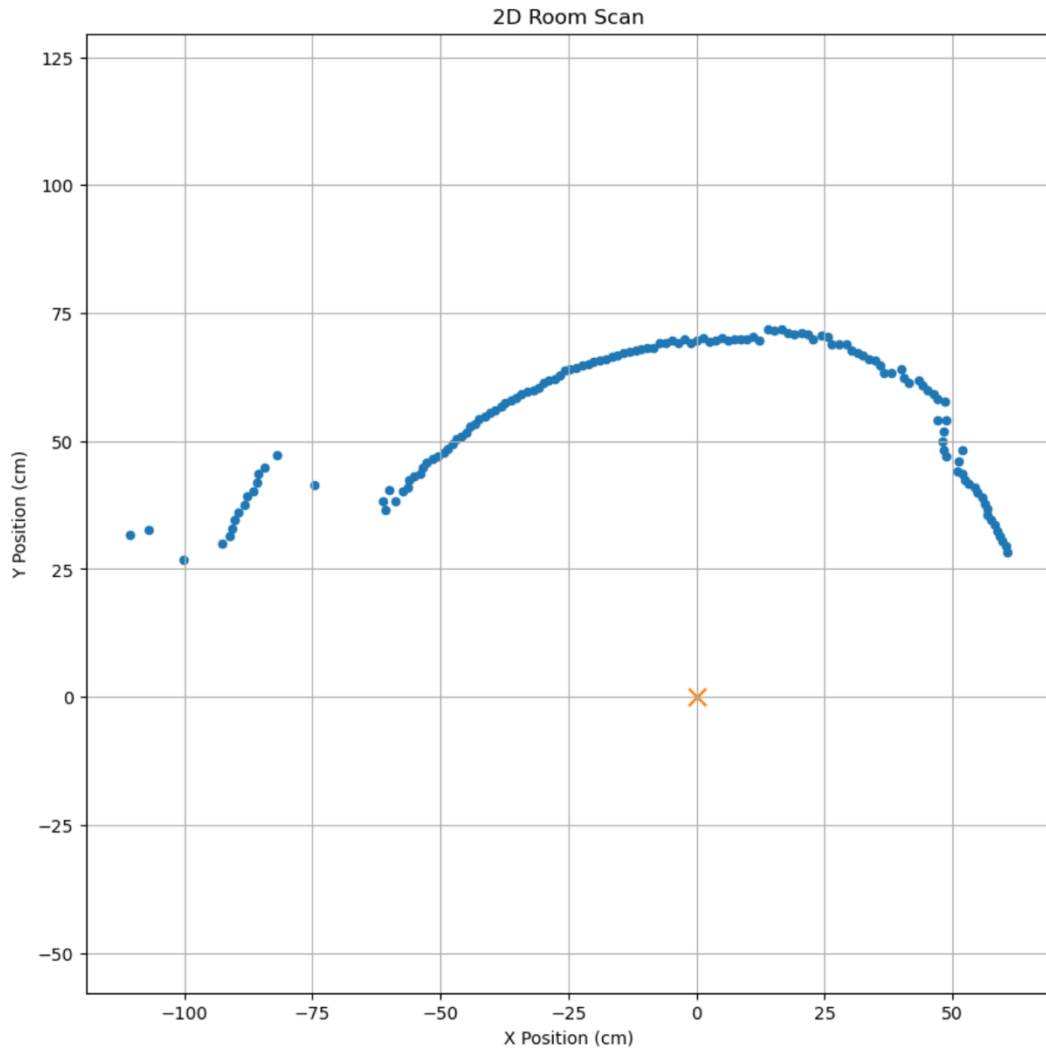


Figure 2

Further issues were phantom points at physically impossible distances, and increased noise at the extremes of the sweep where the beam strikes surfaces at glancing angles. These limitations drove the development of software filtering attempts discussed later and ultimately motivated the hardware upgrade to a laser-based time-of-flight sensor also discussed later.

5.5 Code Fix Attempt

To further improve the visualisation and extract a meaningful result from the scan data, an automated wall detection algorithm was attempted in Python. Rather than manually specifying angular ranges for each wall as was initially attempted, the algorithm analysed the distance readings sequentially and identifies walls automatically by detecting sudden jumps in distance between consecutive angle steps using linear regression.

The algorithm, implemented as a `find_segments()` function, worked through the distance readings in sweep order and flags a new segment boundary wherever the change in distance between adjacent readings exceeds a defined threshold, which was set to 5cm by default. This threshold corresponds to the physical transition between two walls meeting at a corner, where the measured distance changes sharply as the sensor beam moves from one surface to another.

Each detected segment, representing a wall surface, was then passed to a linear regression function using `scipy.stats.linregress`. A straight line was fitted through the Cartesian coordinates of the segment points and the R^2 value was used to assess the quality of the fit. Segments with an R^2 above 0.5 were considered sufficiently linear to represent a real wall and a fitted line was drawn through them on the visualisation.

This approach's goal was to upgrade the visualisations produced without upgrading the hardware, and make a cleaner, more interpretable visualisation of the room's corner than the raw point cloud alone.

This yielded the following in Figure 3:

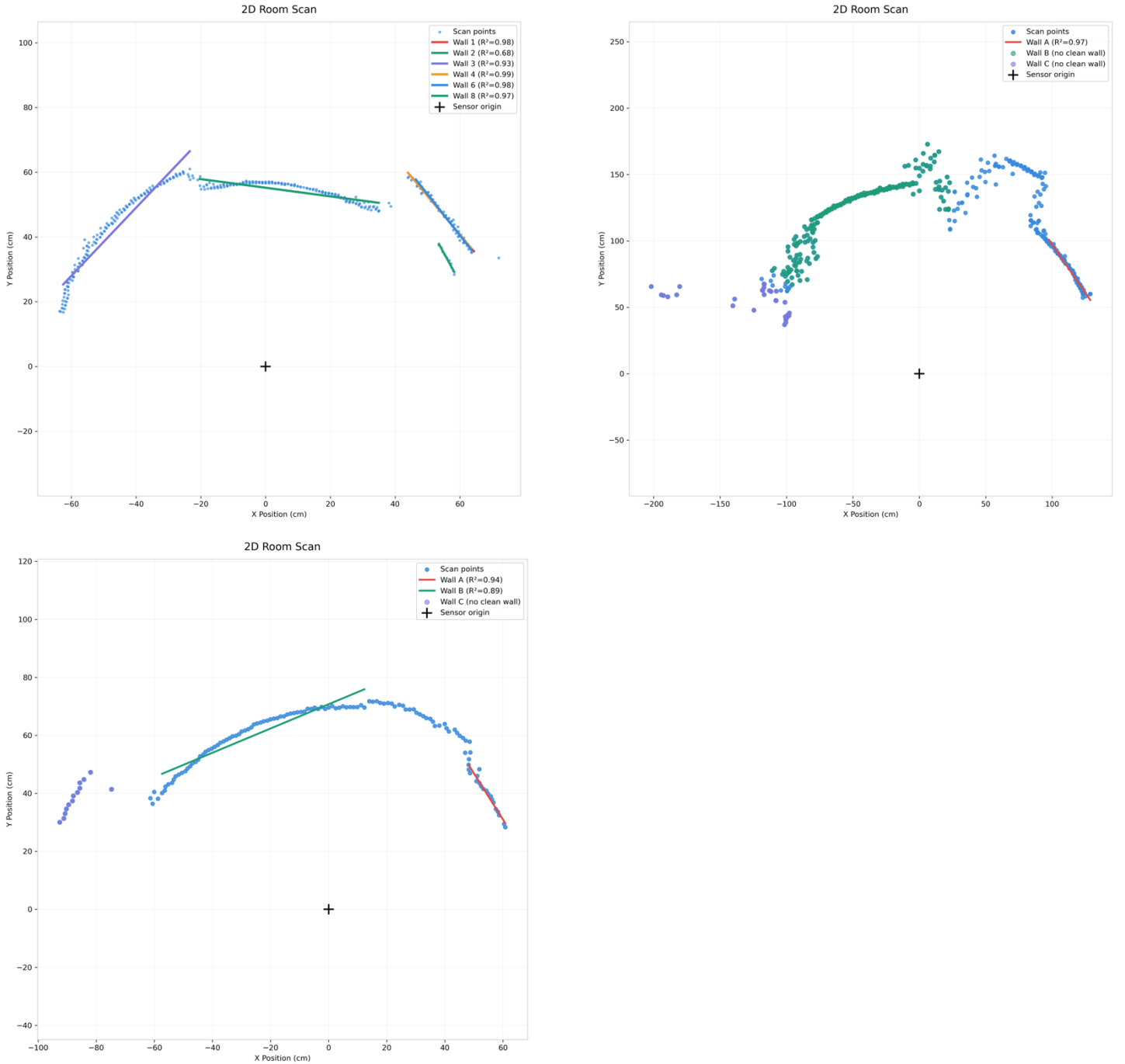


Figure 3

The automated wall detection algorithm produced mixed results across different scans. In the best case, the algorithm successfully identified multiple wall segments with high R^2 values, with Wall 1, Wall 4 and Wall 6 all achieving R^2 values above 0.97 (top left), indicating strong linear fits. However, in scans shown the algorithm struggled and instead detected noisy or fragmented segments that did not clearly correspond to real wall surfaces, and in some cases only fitting a single clean wall line despite multiple walls being present in the scan. These inconsistent results highlighted the limitations of the HC-SR04 data quality as the primary constraint on the wall detection performance, rather than the algorithm itself.

6. TF-Luna LiDAR Range Finder – Second Prototype

Using the exact same set-up but a far better sensor, the TF-Luna LiDAR sensor was identified as a suitable upgrade from the HC-SR04. However, the sensor was more expensive and less available for UK shipping. The new sensor offered a significantly narrower 2° beam width compared to the 15° cone of the ultrasonic sensor, a maximum range of 8m, and laser-based time-of-flight measurement for improved accuracy. The sensor communicates via UART serial interface and was intended to connect directly to the ESP32 using the hardware serial pins.

However, upon initial connection the sensor began smoking within seconds of being powered on and was immediately disconnected. Subsequent testing confirmed the sensor was unresponsive and not functional. The cause was likely to be a wiring fault during the initial connection, most likely a short circuit caused by an incorrect pin connection. This was an important lesson in the importance of thoroughly verifying wiring before powering on new hardware components.

The failure of the TF-Luna, combined with shipping delays and import costs associated with replacing it, led to the decision to instead investigate alternative laser-based sensors available from UK suppliers. This ultimately led to the selection of the VL53L0X time-of-flight sensor as the replacement.

Figure 4 shows an image of the sensor set up of TF-Luna LiDAR but non-functional, Figure 5 is an image of the front of the sensor:

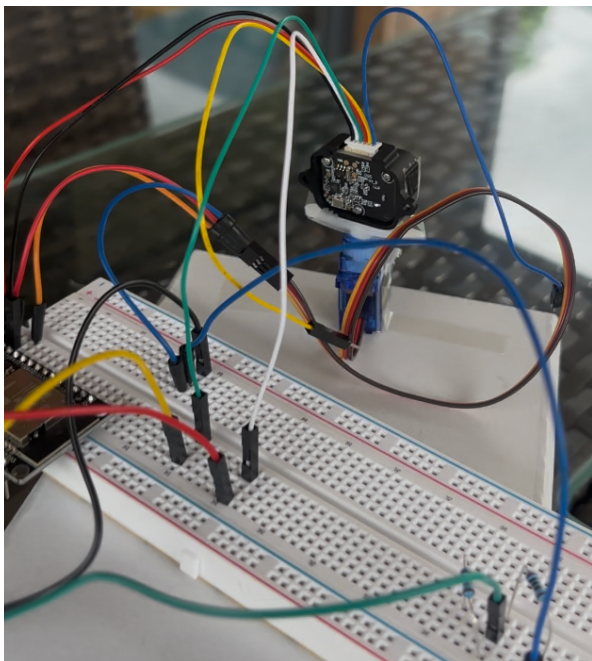


Figure 4



Figure 5

7. VL53L0X V2 Time-of-Flight Laser Distance Sensor – Final Prototype

7.1 Wiring and Hardware Setup

The VL53L0X sensor was chosen partly due to its simpler and safer wiring compared to the TF-Luna. It runs at 3.3V, the same voltage as the ESP32, eliminating the voltage mismatch that had caused the TF-Luna to fail, and only requires four wires to connect, making it straightforward to set up.

For the set up for this sensor, there was soldering needed to be completed. Figure 6 shows the unsoldered sensor as it arrived. Figure 7 shows the soldered sensor ready for use:

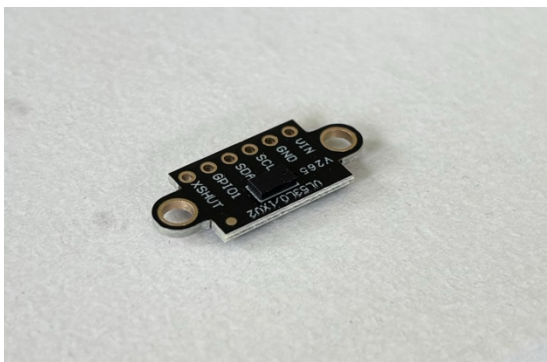


Figure 6

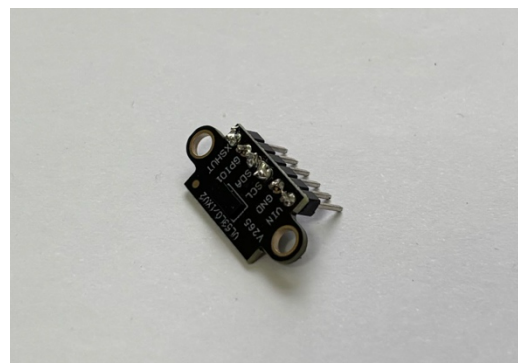


Figure 7

The sensor was then again mounted to the servo as shown below in Figure 8:

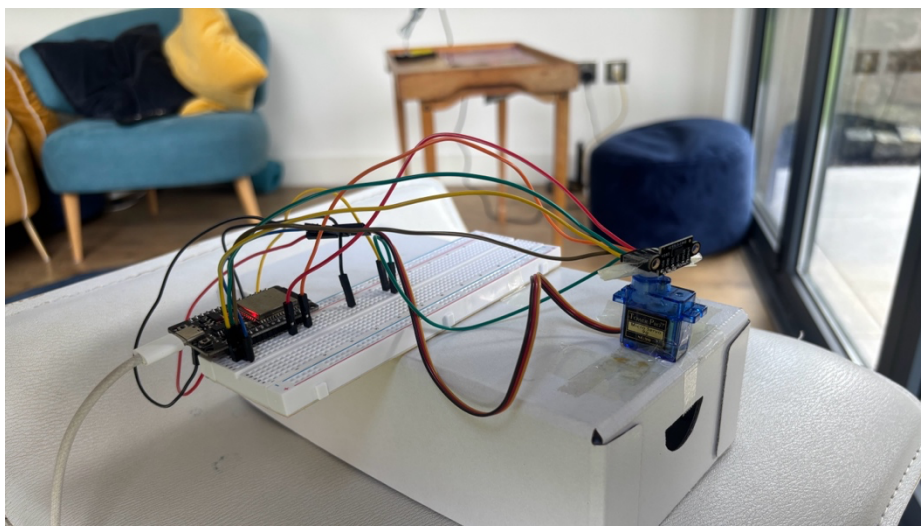


Figure 8

7.2 Arduino Code

The Arduino code needed updating to work with the new sensor. The main change was how the distance reading is obtained, the HC-SR04 required firing a sound pulse and timing its return, whereas the VL53L0X simply returns a distance value directly each time it is asked, making the code simpler and cleaner.

The new sensor also automatically flags any reading it considers unreliable, so these could be discarded without any additional manual checking in the code.

Since the VL53L0X produces more consistent readings than the HC-SR04, the number of samples taken at each angle was reduced from 10 to 5 and the waiting time between steps was reduced, speeding up the overall scan without sacrificing data quality.

Importantly the data sent over serial remained in the same format as before, meaning the Python scripts used to collect and visualise the data required no changes at all.

Here is the full code used:

```
#include <Wire.h>
#include <Adafruit_VL53L0X.h>
#include <ESP32Servo.h>

#define SERVO_PIN 13

Servo scannerServo;
Adafruit_VL53L0X sensor = Adafruit_VL53L0X();

const int startAngle = 25;
const int endAngle = 165;
const int stepAngle = 1;
const int settleDelay = 150;
const int samplesPerAngle = 5;

float averageReading() {
    float total = 0;
    int validCount = 0;

    for (int i = 0; i < samplesPerAngle; i++) {
        VL53L0X_RangingMeasurementData_t measure;
        sensor.rangingTest(&measure, false);

        if (measure.RangeStatus != 4) {
            float d = measure.RangeMilliMeter / 10.0; // mm to cm
            if (d > 5 && d < 200) {
                total += d;
                validCount++;
            }
        }
    }
}
```

```

        delay(20);
    }

    if (validCount == 0) return -1;
    return total / validCount;
}

void setup() {
    Serial.begin(115200);
    Wire.begin(21, 22);
    delay(1000);

    if (!sensor.begin()) {
        Serial.println("Sensor not found - check wiring");
        while (1);
    }

    Serial.println("Sensor ready");

    scannerServo.attach(SERVO_PIN);
    delay(1000);

    Serial.println("angle,distance");
}

void loop() {
    for (int angle = startAngle; angle <= endAngle; angle += stepAngle) {
        scannerServo.write(angle);
        delay(settleDelay);

        float distance = averageReading();

        if (distance > 0) {
            Serial.print(angle);
            Serial.print(",");
            Serial.println(distance);
        }
    }

    Serial.println("SCAN_COMPLETE");
    delay(3000);
}

```

7.3 Python Code

The Python data collection and visualisation scripts required no changes when changing from the HC-SR04 to the VL53L0X sensor. Since the serial output format was kept the same with the angle and distance in centimetres separated by a comma making the existing PySerial logging script and Matplotlib visualisation work without change.

This was a deliberate decision made early in the project, by keeping the data format the same regardless of which sensor was being used, the Python scripts doesn't need to be rewritten when the hardware was upgraded. This saved time during the process.

7.4 Results

The transition to the VL53L0X time-of-flight sensor produced a noticeable improvement in scan quality compared to the HC-SR04 ultrasonic sensor. As shown in Figure 9 & 10, the point cloud is significantly tighter and more consistent, with wall surfaces appearing as dense well-defined clusters of points rather than the scattered arc shapes seen in the ultrasonic scans and a definite corner can be identified as seen below in Figure 9 & 10.

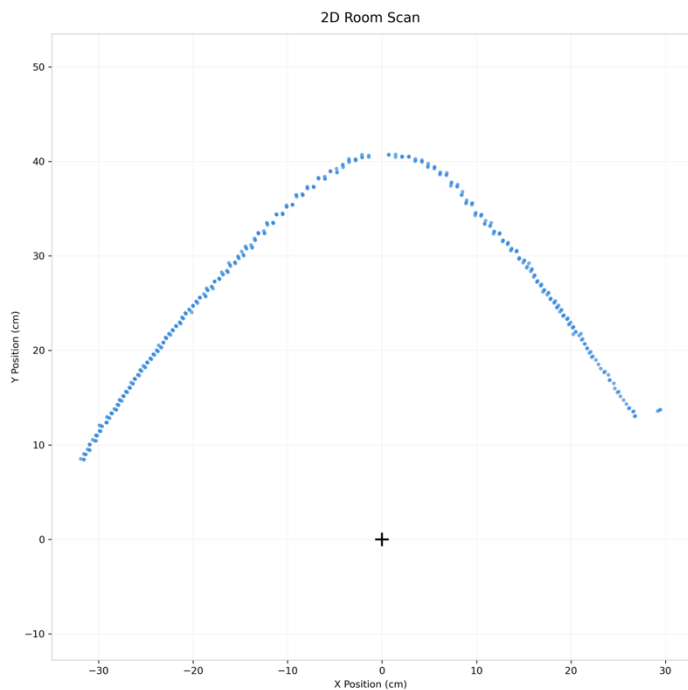


Figure 9

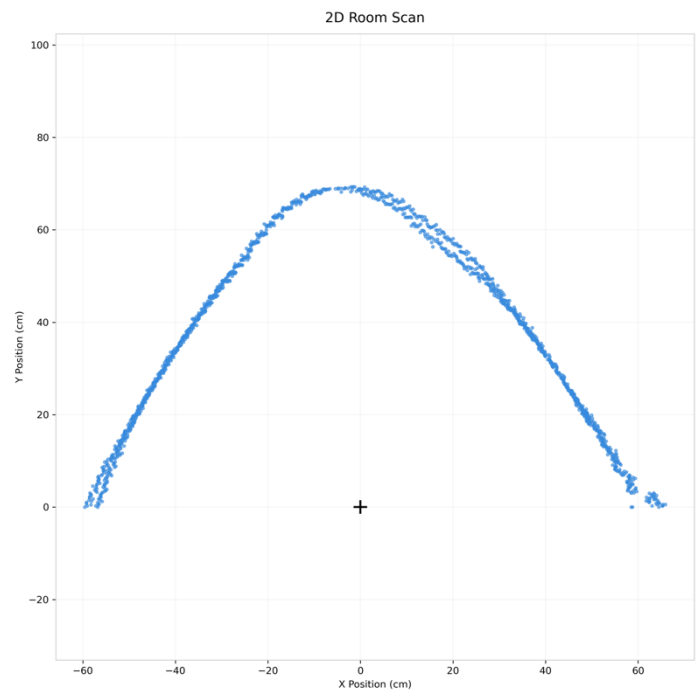


Figure 10

The narrower 3° beam width of the laser sensor compared to the 15° cone of the HC-SR04 resulted in more precise individual readings, with corner regions appearing sharper and more distinct. The multipath reflection issues that had produced phantom points at impossible distances in the HC-SR04 scans were largely eliminated, with the laser-based measurement proving far less susceptible to indirect reflections.

Taking advantage of the improved data consistency, the sweep resolution was increased from 1° steps (Figure 8) to 0.25° steps (Figure 9), producing 720 data points across the 180° sweep compared to the original 140. This produced a noticeably denser point cloud as shown in Figure 9 & 10, giving a clearer representation of room surfaces and making wall boundaries more distinct.

Overall, the VL53L0X results represented a significant improvement in scan quality, confirming the decision to upgrade from ultrasonic to laser-based sensing and providing a strong foundation for development for a future 3D scanning system.

8. Further Development – 3D Room Reconstruction – First Prototype

8.1 Wiring and Hardware Setup

To extend the system from 2D to 3D scanning, a second SG90 servo motor was added to control the vertical tilt angle of the sensor. This second servo was mounted on its side and oriented to tilt the sensor assembly up and down between horizontal sweeps, with the original horizontal scanning servo mounted on top and carrying the sensor.

Below are Figures 11 & 12 showing the 3D setup:

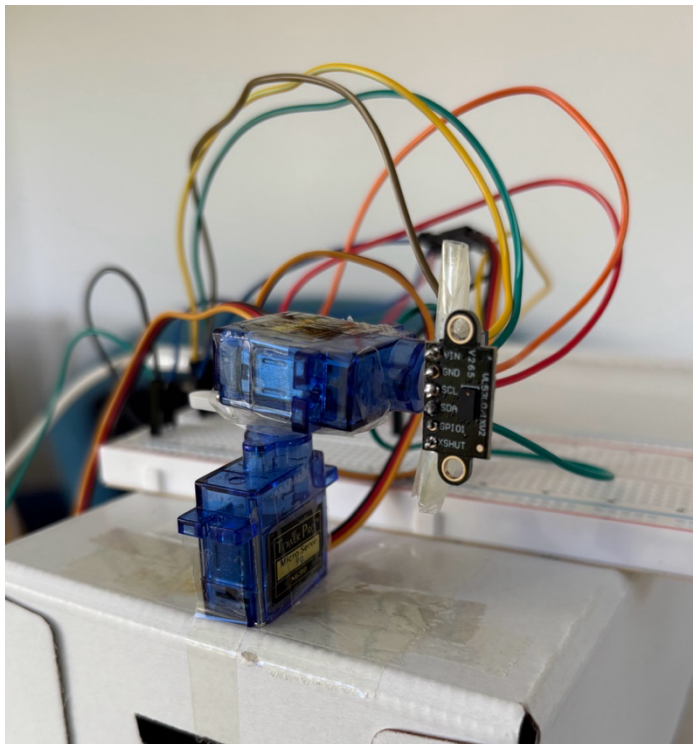


Figure 11



Figure 12

8.2 Arduino Code

The Arduino code was extended from the 2D system to add the second servo and produce a full 3D scan. The fundamental structure remained the same, the sensor is activated at each servo position and the result transmitted over serial, but a second loop was added to control the vertical tilt position of the second Servo between each horizontal sweep. The overall goal for this prototype is the same, to try and visualise a corner of a room.

The code was structured so that the tilt servo moves to a new vertical position then triggers a new full horizontal sweep. This is repeated at each vertical position, building up a series of layers that together form the 3D point cloud.

The data sent over serial was updated to include three values: the horizontal angle, the vertical tilt angle, and the distance. This is so that the Python script could correctly position each point in 3D space. Two progress markers were also added so the Python script could track which layer was being scanned and automatically stop recording when the full scan was complete.

Each full 3D scan covered 7 vertical layers and took approximately 13 minutes to complete.

Here is the full code used:

```
#include <Wire.h>
#include <Adafruit_VL53L0X.h>
#include <ESP32Servo.h>

#define SERVO_PIN 13
#define SERV02_PIN 12

Servo scannerServo;
Servo tiltServo;
Adafruit_VL53L0X sensor = Adafruit_VL53L0X();

const int startAngle = 0;
const int endAngle = 140;
const float stepAngle = 0.5;
const int settleDelay = 100;
const int samplesPerAngle = 3;

const int tiltStart = 0;    // servo position for lowest tilt
const int tiltEnd = 60;    // servo position for highest tilt
const int tiltStep = 10;   // degrees between each horizontal sweep

float averageReading() {
    float total = 0;
    int validCount = 0;
```

```

for (int i = 0; i < samplesPerAngle; i++) {
    VL53L0X_RangingMeasurementData_t measure;
    sensor.rangingTest(&measure, false);

    if (measure.RangeStatus != 4) {
        float d = measure.RangeMilliMeter / 10.0;
        if (d > 5 && d < 200) {
            total += d;
            validCount++;
        }
    }
    delay(20);
}

if (validCount == 0) return -1;
return total / validCount;
}

void setup() {
    Serial.begin(115200);
    Wire.begin(21, 22);
    delay(1000);

    if (!sensor.begin()) {
        Serial.println("Sensor not found - check wiring");
        while (1);
    }

    Serial.println("Sensor ready");

    scannerServo.attach(SERVO_PIN);
    tiltServo.attach(SERV02_PIN);

    // Move both servos to start position
    scannerServo.write(startAngle);
    tiltServo.write(tiltStart);
    delay(1000);

    Serial.println("angle,tilt,distance");
}

void loop() {
    for (int tilt = tiltStart; tilt <= tiltEnd; tilt += tiltStep) {
        // Move tilt servo to next position
        tiltServo.write(tilt);
        delay(500);

        // Convert servo position to actual tilt angle for logging
        int tiltAngle = tilt - 90; // centres around 0° at servo position 90

        // Do full horizontal sweep at this tilt
    }
}

```

```

for (float angle = startAngle; angle <= endAngle; angle += stepAngle) {
    scannerServo.write(angle);
    delay(settleDelay);

    float distance = averageReading();

    if (distance > 0) {
        Serial.print(angle);
        Serial.print(",");
        Serial.print(tiltAngle);
        Serial.print(",");
        Serial.println(distance);
    }
}

Serial.print("SWEEP_COMPLETE_");
Serial.println(tiltAngle);
}

Serial.println("SCAN_COMPLETE");
delay(5000);
}

```

8.3 Python Code

The Python data collection script was updated to handle the new three column serial output format, parsing each incoming line into angle, tilt and distance values and writing all three to the file created containing the data. The script was also updated to handle the SWEEP_COMPLETE and SCAN_COMPLETE markers transmitted by the Arduino, printing progress updates to the console and automatically stopping data collection when the full 3D scan was complete.

The visualisation script required more significant changes to accommodate the 3D nature of the data. Rather than converting polar coordinates to 2D Cartesian X and Y positions, a full 3D conversion was applied using both the horizontal sweep angle and the vertical tilt angle of each reading:

```
x = distance * cos(horizontal_angle) * cos(tilt_angle)
y = distance * sin(horizontal_angle) * cos(tilt_angle)
z = distance * sin(tilt_angle)
```

The rolling median filter was applied separately to each horizontal tilt slice before the 3D conversion, ensuring that outlier removal was performed within each sweep rather than across the entire dataset. Plotly was used in place of Matplotlib for the 3D visualisation. The final visualisation was saved as an HTML file for easy viewing. Other than these changes the Python script was kept the same.

8.4 Results

The first 3D scan produced the point cloud shown in Figure 13 below, with the seven horizontal slices clearly visible as distinct layers coloured from dark red at the lowest tilt angle through to yellow at the highest. The layered structure of the scan is immediately noticeable, showing that the double servo system successfully captured data at multiple vertical angles across the full 180° horizontal sweep, successfully capturing a 3D scan of a corner.

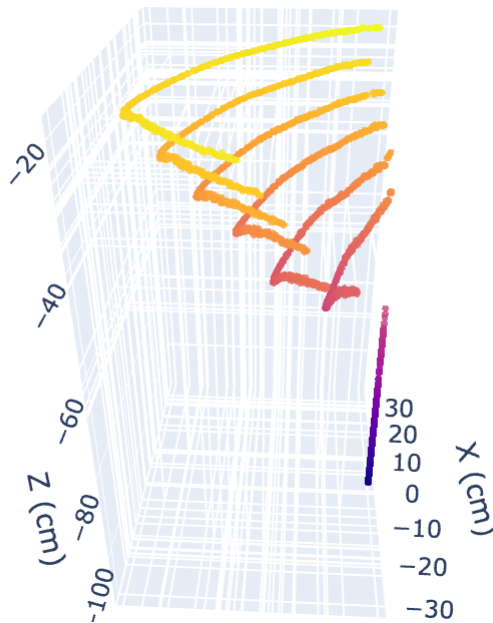


Figure 13

However, the result highlights a slight geometric challenge. The tilt range of 60° meant the sensor was scanning predominantly downward rather than outward, causing the point cloud to appear heavily compressed in the vertical axis and stretched in the Z direction. This produced a distorted image of the room that doesn't correspond exactly to the expected shape of the scanned space.

Additionally, the polar coordinate effect observed in the 2D scans where flat walls appear as curved arcs is also in the 3D visualisation by the vertical tilt angles, distorting the apparent shape of the point cloud slightly. At steeper tilt angles the sensor beam travels a longer path to reach the same wall surface, producing readings that appear further away than the true perpendicular distance.

These results identified two clear areas for improvement: adjusting the physical tilt range to scan closer to horizontal and refining the coordinate conversion to better account for the geometry of the dual servo system.

9. Conclusion and Reflections

This project set out to demonstrate that meaningful room scanning, and 3D spatial mapping could be achieved using low-cost hobbyist electronics, without relying on expensive professional lidar equipment. Through an iterative hardware and software development process, this goal was successfully achieved.

The progression from HC-SR04 ultrasonic sensing to VL53L0X time-of-flight laser sensing represented the most significant hardware improvement of the project, directly addressing the limitations that made the quality of the initial scan's poor. The introduction of a second servo motor to control vertical tilt successfully extended the system into three dimensions, producing a multi-layer point cloud that demonstrates the viability of the dual servo scanning approach.

The project also highlighted the practical challenges of working with low-cost hardware; from the TF-Luna failure to soldering difficulties and library compatibility issues, all of which required systematic diagnosis and problem solving to resolve. These challenges are reflective of real engineering development processes and contributed as much to the learning outcomes of the project as the successful results themselves.

With a total hardware cost of under £30, the system represents a viable proof of concept for low-cost room mapping and establishes a strong foundation for future development in a more accurate and complete 3D room reconstruction system.